```
!pip install kaggle
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (1.7.4.5)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)
Requirement already satisfied: certifi>=14.05.14 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2026.2
Requirement already satisfied: charset-normalizer in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.4.4
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from kaggle) (3.11)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.6)
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: setuptools>=21.0.0 in /usr/local/lib/python3.12/dist-packages (from kaggle) (75.2.
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.17.0)
Requirement already satisfied: text-unidecode in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.3)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.5.1)
```

Load the **dataset**

```
import pandas as pd

# replace filename with your uploaded file
df = pd.read_csv("covid.csv")

# inspect data
df.head()
df.info()
df.describe()
df.isnull().sum()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 149301 entries, 0 to 149300
Data columns (total 67 columns):
 #   Column                                      Non-Null Count   Dtype
---  ------                                      --------------   -----
 0   iso_code                                    149301 non-null  object
 1   continent                                   142356 non-null  object
 2   location                                    149301 non-null  object
 3   date                                        149301 non-null  object
 4   total_cases                                 138871 non-null  float64
 5   new_cases                                   146592 non-null  float64
 6   new_cases_smoothed                          146057 non-null  float64
 7   total_deaths                                131092 non-null  float64
 8   new_deaths                                  146615 non-null  float64
```

Clean and transform

```
# convert date
df['date'] = pd.to_datetime(df['date'])
# handle misiing data
df.fillna(0, inplace=True)  # replaces NaN with 0
```

```
 16  reproduction_rate                           80901 non-null   float64
 17  icu_patients                                17242 non-null   float64
 18  icu_patients_per_million                    17242 non-null   float64
 19  hosp_patients                               15567 non-null   float64
 20  hosp_patients_per_million                   15567 non-null   float64
```

Explore and Visualize

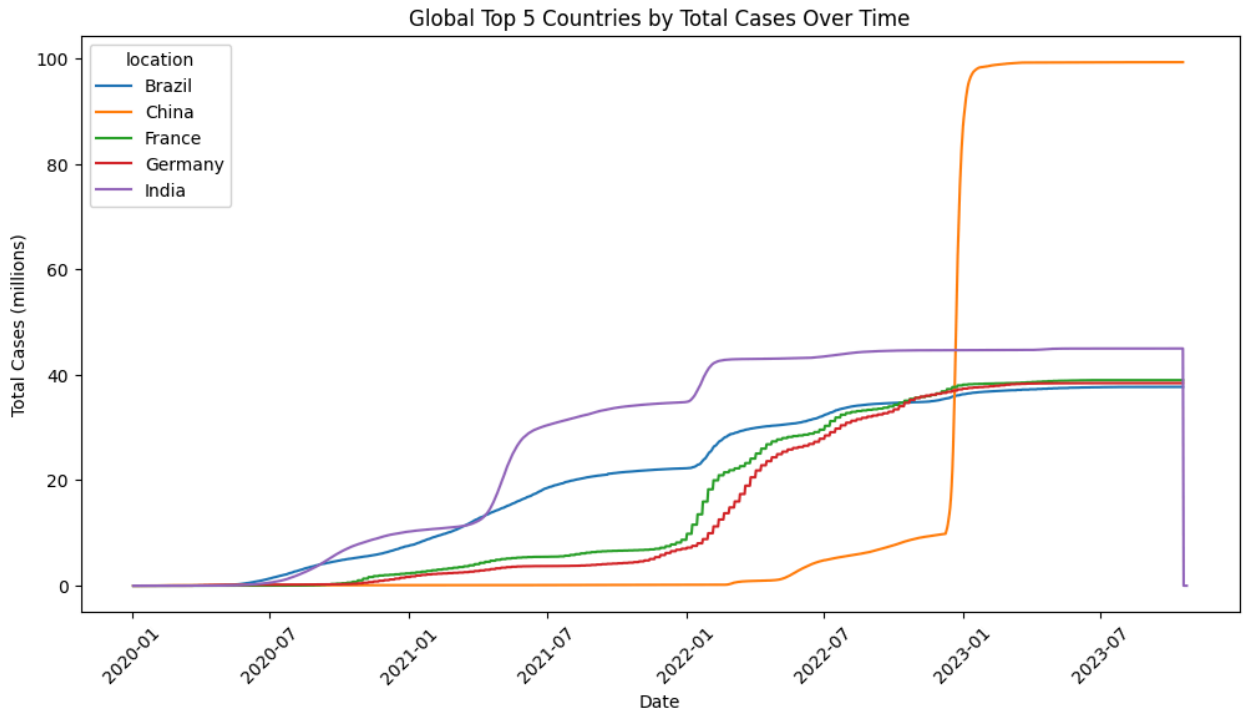
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ensure 'date' is datetime
df['date'] = pd.to_datetime(df['date'])

# convert numeric columns
df['total_cases'] = pd.to_numeric(df['total_cases'], errors='coerce').fillna(0)
df['total_deaths'] = pd.to_numeric(df['total_deaths'], errors='coerce').fillna(0)
df['total_vaccinations'] = pd.to_numeric(df['total_vaccinations'], errors='coerce').fillna(0)

# remove aggregate rows
aggregates = ['Asia','Europe','Africa','Oceania','High income','Low income','European Union','World']
df_countries = df[df['continent'].notna() & (~df['location'].isin(aggregates))]

# global top 5 countries by total cases
top5_global = df_countries.groupby('location')['total_cases'].max().sort_values(ascending=False).head(5).index
df_top5_global = df_countries[df_countries['location'].isin(top5_global)]

plt.figure(figsize=(12,6))
sns.lineplot(data=df_top5_global, x='date', y=df_top5_global['total_cases']/1e6, hue='location')
plt.title("Global Top 5 Countries by Total Cases Over Time")
plt.xlabel("Date")
plt.ylabel("Total Cases (millions)")
plt.xticks(rotation=45)
plt.show()
# aggregate deaths globally (only real countries)
global_deaths = df_countries.groupby('date')['total_deaths'].sum().reset_index()

plt.figure(figsize=(12,6))
sns.lineplot(data=global_deaths, x='date', y='total_deaths')
plt.title("Global COVID-19 Total Deaths Over Time")
plt.xlabel("Date")
plt.ylabel("Total Deaths")
plt.xticks(rotation=45)
plt.show()

# function to plot top 5 per continent
def plot_top5_continent(continent_name):
    df_continent = df_countries[df_countries['continent'] == continent_name]
    top5_cont = df_continent.groupby('location')['total_cases'].max().sort_values(ascending=False).head(5).index
    df_top5_cont = df_continent[df_continent['location'].isin(top5_cont)]

    plt.figure(figsize=(12,6))
    sns.lineplot(data=df_top5_cont, x='date', y=df_top5_cont['total_cases']/1e6, hue='location')
    plt.title(f"Top 5 Countries in {continent_name} by Total Cases")
    plt.xlabel("Date")
    plt.ylabel("Total Cases (millions)")
    plt.xticks(rotation=45)
    plt.show()

# Example: plot top 5 in Asia and Europe
plot_top5_continent('Asia')
plot_top5_continent('Europe')
```

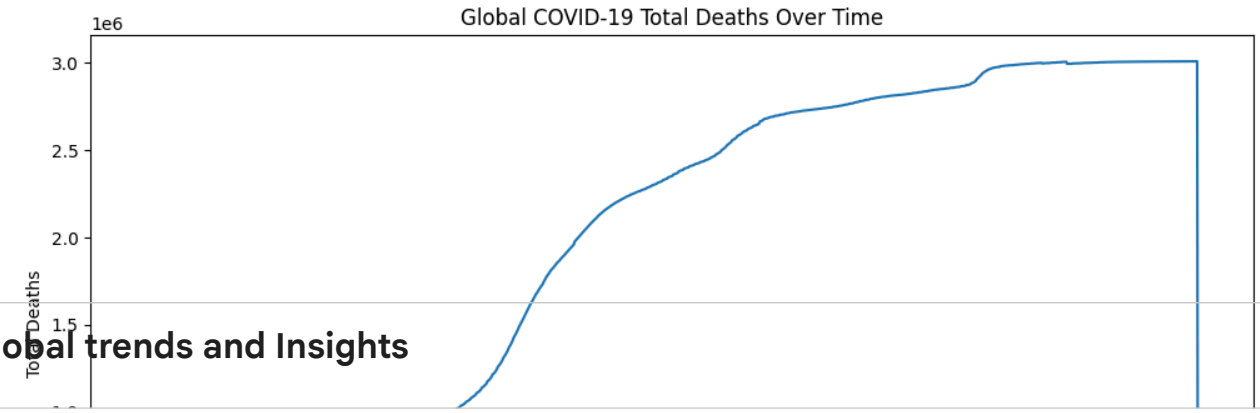
population	1
excess_mortality_cumulative_absolute	143739

excess_mortality_cumulative	143739
excess_mortality	143739
excess_mortality_cumulative_per_million	143739

67 rows × 1 columns

dtype: int64





Global trends and Insights

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# convert date and numeric columns
df['date'] = pd.to_datetime(df['date'])
numeric_cols = ['total_cases','total_deaths','new_cases','new_deaths','total_vaccinations']
for col in numeric_cols:
    if col not in df.columns:
        df[col] = 0
    df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0)

# keep only real countries (remove aggregates)
aggregates = ['Asia','Europe','Africa','Oceania','High income','Low income','European Union','World']
df_countries = df[df['continent'].notna() & (~df['location'].isin(aggregates))]

# global totals
global_total_cases = df_countries['total_cases'].max()
global_total_deaths = df_countries['total_deaths'].max()
global_total_vaccinations = df_countries['total_vaccinations'].max()
print(f"Global Total Cases: {int(global_total_cases):,}")
print(f"Global Total Deaths: {int(global_total_deaths):,}")
print(f"Global Total Vaccinations: {int(global_total_vaccinations):,}")

# daily totals
global_daily = df_countries.groupby('date')[['total_cases','total_deaths','total_vaccinations']].sum(numeric_only=

# plot global trends
plt.figure(figsize=(12,6))
sns.lineplot(data=global_daily, x='date', y=global_daily['total_cases']/1e6, label='Total Cases')
sns.lineplot(data=global_daily, x='date', y=global_daily['total_deaths']/1e6, label='Total Deaths')
if 'total_vaccinations' in global_daily.columns:
    sns.lineplot(data=global_daily, x='date', y=global_daily['total_vaccinations']/1e6, label='Total Vaccinations
plt.title("Global COVID-19 Trends Over Time")
plt.xlabel("Date")
plt.ylabel("Count (millions)")
plt.xticks(rotation=45)
```